

Characterization of Large-scale HPC Workloads with non-naïve I/O Roofline Modeling and Scoring

Zhaobin Zhu* and Sarah Neuwirth^{†‡}

*Goethe University Frankfurt, Germany, zhu@em.uni-frankfurt.de

[†]Johannes Gutenberg University Mainz, Germany, neuwirth@uni-mainz.de

[‡]Jülich Supercomputing Centre, Forschungszentrum Jülich GmbH, Germany

Abstract—This paper introduces a novel approach to characterize system and application performance in high performance computing (HPC) systems. Traditional metrics such as computations and memory accesses alone are no longer sufficient to evaluate the performance of such systems. To address this challenge, an empirical I/O Roofline model and corresponding workload analysis workflow are proposed that can be adapted in the future to enable multidimensional evaluation for application performance characterization across different HPC systems. The model focuses on commonly used performance metrics such as I/O operations per second (IOPS) and I/O bandwidth, and leverages the well-known Roofline modeling technique to intuitively characterize I/O performance and identify performance bottlenecks without requiring deep knowledge of the I/O stack. Furthermore, based on the I/O Roofline model, a scoring approach is described that provides a unified and comprehensible method for evaluating the performance of different systems and applications. The effectiveness of the approach is demonstrated by evaluating the performance of various application I/O kernels using the empirical Roofline model.

Index Terms—I/O, HPC, Performance Modeling Workflow, I/O Analysis, I/O Performance, I/O System Evaluation.

I. INTRODUCTION

In the era of exascale computing and the emergence of new concepts such as modular supercomputing architecture (MSA) [1], where different modules and technologies are combined into a single workflow, the landscape of scientific applications is undergoing a drastic change. This shift is leading to multi-level, multi-dimensional workloads across different software layers and hardware components [2], [3]. Typically, different applications result in different I/O patterns and requirements. Depending on the application, the access patterns, data volumes, sequentiality, and parallelism of I/O operations vary significantly [4]. Computational units are optimized for specific applications and computational power increases; at the same time, the diversity of workloads, and the heterogeneity of the systems increase, making performance modeling a challenging task. Furthermore, as HPC systems grow in size and new workloads emerge, workloads generally become more data-intensive, leading to even less predictable I/O behavior and access patterns [5]. Given the widening gap between computational performance and I/O capacity, modeling techniques need to be intuitive so that people with no prior expertise can understand the observed performance [6]–[8]. Meanwhile, these techniques should be adaptable to comprehensively assess performance across

multiple factors, including memory, I/O, and networking of applications and systems [9], [10]. Due to the lack of unified performance modeling techniques for I/O, and the ability to evaluate systems and applications across multiple performance factors, we propose an extension of the naïve Roofline model to also include I/O characterization and make the following contributions:

- An **empirical Roofline model** that can be used to analyze I/O workload at a large scale and provide the foundation for a multidimensional assessment (i.e., score) for system and application characterization.
- A **scoring approach** that can be applied to derive a meaningful score based on our I/O Roofline model which can be used to rank different systems and applications in terms of their I/O performance.
- A **generic workflow** with a corresponding tool for the Roofline analysis, i.e., system-specific performance can be derived automatically from community benchmarks such as IOR [11] and presented as the Roofline in a visualization. Moreover, application-specific I/O performance can be extracted from Darshan [12] outputs and the model is automatically visualized.

II. PERFORMANCE MODELING AND ROOFLINE MODEL

Performance modeling is a structured technique and offers a comprehensive understanding of system performance from design through the life cycle [13]. Common approaches include queueing theory, discrete-event simulation, rule-based approaches, analytical modeling, and machine learning [13]–[18]. A rule-based method is *bound and bottleneck analysis* [19]. Bound analysis identifies performance limitations, while bottleneck analysis focuses on the component with the longest processing time, both serving to optimize system performance by addressing bottlenecks or limiting factors.

The Roofline model [6] is an increasingly popular approach for node-level performance characterization based on bound and bottleneck analysis. This model visualizes performance capabilities and limitations of parallel computing systems, revealing potential bottlenecks. As shown in Figure 1, it plots operational intensity (FLOPS/byte) against achievable performance (FLOPS). Operational intensity, on the x-axis, measures the ratio of floating-point operations to memory bytes accessed, while the y-axis represents the maximum

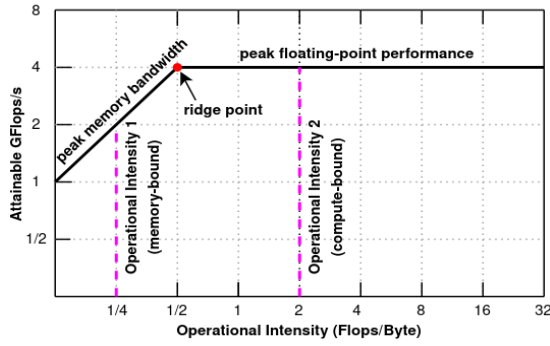


Fig. 1: Example of a naïve Roofline Model.

achievable performance. Equation 1 defines the *maximum attainable performance* for a computer system. The upper performance limit in the classic Roofline model is influenced by system hardware characteristics and constraints, including available memory bandwidth and compute power.

$$\text{Attainable GFlops/sec} = \text{Min}(\text{Peak Floating Point Performance}, \text{Peak Memory Bandwidth} \times \text{Operational intensity}) \quad (1)$$

Another crucial metric for this model is the *operational intensity*, as indicated in Equation 2. This metric reflects the balance between computational and communication work in a given algorithm. Thus, if more memory bytes are transferred per operation, the resulting operating intensity is lower. Conversely, the operating intensity is higher when fewer memory bytes are transferred per operation. To demonstrate operational intensity, Williams et al. [6] characterized four typical floating-point kernels as shown in Table I.

$$\text{Operational intensity} = \frac{\text{Floating point operations}}{\text{Memory bytes transferred}} \quad (2)$$

To differentiate compute capacity or memory bandwidth constraints in an algorithm, imagine a column rising from the x-axis to the Roofline. If it touches the flat Roofline part, it is called *compute-bound*. If it hits the sloped Roofline, it is referred to as *memory-bound*. Typically, the peak performance is determined by system-specific benchmarks such as STREAM [20] or hardware specifications, the operational intensity is determined through algorithmic analysis.

Moreover, the point where the diagonal and horizontal ceilings intersect is known as the ridge point (RP). This point represents the optimal operational intensity for a given system,

Name	Op.Inten.	Description
SpMV [21]	0.17 - 0.25	Sparse Matrix-Vector multiply: $y = A*x$ where A is a sparse matrix and x, y are dense vectors; multiplies and adds equal.
LBMHD [22]	0.70 - 1.07	Lattice-Boltzmann Magnetohydrodynamics is a structured grid code with a series of time steps.
Stencil [23]	0.33 - 0.50	A multigrid kernel that updates 7 nearby points in a 3-D stencil for a 2563 problem.
3-D FFT [24]	1.09 - 1.64	Three-Dimensional Fast Fourier Transform (2 sizes: 1283 and 5123).

TABLE I: Operational Intensity by Algorithms [6].

and can be used to identify potential performance bottlenecks. By analyzing the RP, system architects and developers can optimize their algorithms and improve the overall performance of the system. Overall, the Roofline model is a powerful and intuitive approach to help users with less prior knowledge to understanding performance of their applications and systems.

III. ANALYSIS OF RELATED WORK

The IO500 benchmark [25] is the standard for evaluating parallel file system performance. It uses predefined workloads and I/O patterns, including small and large file operations, sequential and random patterns, and metadata operations, for a comprehensive assessment. Based on underlying test cases, the benchmark produces a single IO500 score for easy system performance comparison. However, its score's significance is debated. It combines different benchmarks with various units (GiB/s, KIOPS), creating a score unit questioned for its validity. IO500 also neglects real-world scenarios and is affected by external factors (e.g., network latency).

Liem et al. [26] introduced a visual approach that leverages the IO500 benchmark to predict I/O performance and offer tailored optimization strategies. This method creates a two-dimensional bounding box based on the IO500 best and worst test cases to provide user expectations. By comparing an application's I/O performance with the bounding box, it enables tasks like performance evaluation and anomaly detection. Nevertheless, it comes with some weaknesses. The dependence on IO500 and purely visual analysis based on the position of the bounding box may affect accuracy and applicability to real workloads.

Cardwell and Song [10] extend the Roofline performance model to include communication costs in distributed-memory HPC systems. The communication-aware Roofline model calculates the communication arithmetic intensity and determines attainable performance considering communication bandwidth. The *Ping-Pong* micro-benchmark determines peak communication performance. By considering the communication cost as an additional dimension, the extended Roofline enables the creation of both 2D and 3D visualizations. The *Ridgeline model* [27] simplifies Roofline diagrams for multiple resources into an intuitive planar representation.

IV. EMPIRICAL I/O ROOFLINE MODEL

As HPC systems continue to evolve, emerging workloads place ever-increasing demands on I/O performance. Therefore, it is becoming increasingly important to be able to accurately assess I/O performance. We adopt the intuitive Roofline model for I/O workload analysis, which is an ideal tool for understanding how closely observed I/O performance corresponds to peak performance. In addition, our approach is designed to create a consistent scoring model. Using a standardized approach for assessing I/O performance makes it easier to compare and evaluate different systems and applications.

POSIX	Symbol	MPI-IO	Symbol
OPENS	P _{opens}	INDEP. OPENS	Mi _{opens}
FILENOS	P _{filenos}	COLL. OPENS	Mc _{opens}
DUPS	P _{dups}	INDEP. READS	Mi _{reads}
READS	P _{reads}	INDEP. WRITES	Mi _{writes}
WRITES	P _{writes}	COLL. READS	Mc _{reads}
SEEKS	P _{seek}	COLL. WRITES	Mc _{writes}
STATS	P _{stats}	SPLIT. READS	Ms _{reads}
MMAPS	P _{mmaps}	SPLIT. WRITES	Ms _{writes}
FSYNCS	P _{fsyncs}	NB. READS	Mn _{reads}
FDSYNCS	P _{fdsyncs}	NB. WRITES	Mc _{writes}
		SYNCS	M _{syncs}

TABLE II: Operations for POSIX and MPIIO.

A. Definition of the I/O Roofline Model

In Section III, we emphasized the Roofline model’s flexibility through customized metrics, serving diverse application requirements. It explores the work-traffic relationship, where work quantifies computational effort, and traffic represents the data needed. This perspective extends performance evaluation beyond individual nodes, encompassing system-level analysis. The I/O Roofline model maintains consistent parameters and workflows for both node-level and system-level analysis, making it ideal for holistic I/O performance assessment.

HPC workloads include various applications from different scientific domains, so optimal performance does not require an application to be both compute-intensive and highly I/O-intensive. Thus, when characterizing I/O performance, considering floating point operations for I/O bandwidth is not appropriate. However, to account for both overlap and interactions between different performance factors such as computation and I/O while avoiding bias in the representation of the Roofline model, we plan to apply separate Rooflines for each performance dimension, allowing individual scores that can be aggregated into a multidimensional score.

The I/O Roofline model is based on the most widely accepted I/O performance metric, input/output operations per second (IOPS) and the corresponding bandwidth. IOPS measures the storage system’s reads and writes in a given time, crucial for operations with many small operations. We focus on two popular I/O interfaces in HPC systems: POSIX and MPI-IO. Table II lists the operations used for IOPS calculations across various layers of the I/O stack. Throughput measures the total amount of data that can be read or written per second. It is critical for applications that require large amounts of data to be transferred quickly, such as scientific simulations and machine learning training phases.

In our case, we define the attainable performance, as shown in Eq. 3, as the minimum of the peak IOPS (ψ) or the product of the peak I/O bandwidth (κ) and the I/O intensity (λ).

$$\text{Attainable Performance} = \text{Min}(\psi, \kappa \cdot \lambda) \quad (3)$$

I/O intensity is a critical factor in the Roofline model, however there are various definitions in the literature. One definition is found in the WorkOut approach [28], which enhances RAID reconstruction performance by offloading I/O workload to idle disks in a storage system. Here it is measured

by the number of I/O requests generated per second. Another definition is found in the Vidya tool [29], which characterizes I/O patterns for optimized data access. The authors define it as the rate of I/O operations requested from storage per unit time. Sun and Lionel [30] present an approach to parallel speedup analysis that considers the I/O intensity of parallel programs. Here, I/O intensity is defined as the ratio of total I/O to computation operations in a parallel program.

As there are different definitions of I/O intensity in the literature, a clear and unified definition is essential for the application of the Roofline model. We define I/O intensity as the ratio between the total number of I/O operations (IOP) and the total amount of data movement, as shown in Equation 4:

$$\lambda := \frac{\theta}{\sigma} \quad (4)$$

Since the amount of data moved by an application during execution time is the same for both POSIX and MPI-IO, the total data movement (σ) can be described by Equation 5.

$$\sigma = \sum_{t=1}^n \text{Bytes}_{\text{read}} + \text{Bytes}_{\text{write}} \quad (5)$$

Unlike data movement, file operations differ depending on the I/O interface considered. As can be seen in Table II, the operations can be divided into two subsets: α for POSIX-related operations and β for MPI-IO-specific operations.

$$\alpha = \{P_{\text{opens}}, P_{\text{filenos}}, P_{\text{dups}}, P_{\text{reads}}, P_{\text{writes}}, P_{\text{seek}}, P_{\text{stats}}, P_{\text{mmaps}}, P_{\text{fsyncs}}, P_{\text{fdsyncs}}\} \quad (6)$$

$$\beta = \{Mi_{\text{opens}}, Mc_{\text{opens}}, Mi_{\text{reads}}, Mi_{\text{writes}}, Mc_{\text{reads}}, Mc_{\text{writes}}, Ms_{\text{reads}}, Ms_{\text{writes}}, Mn_{\text{reads}}, Mn_{\text{writes}}, M_{\text{syncs}}\} \quad (7)$$

Since the I/O intensity is determined for the total runtime of the application, the total number of file operations (θ) can be defined as shown in Equation 8. Depending on the I/O interface, the number of operations performed for a given subset, i.e. α or β , is summed up.

$$\theta := \sum_{t=1}^n \left(\sum_{i \in \{\alpha|\beta\}} a_i c_i \right) \quad (8)$$

As each I/O operation incurs different costs, e.g., writes are more resource intensive than reads, we plan to categorize I/O operations by cost in the future. To accurately describe the workloads, we introduce the operation cost constant (c_i), where (a_i) is the number of operations performed. This constant weights each operation based on its cost. As we also focus on the classification of parallel I/O operations, we simplify this work by assuming that each I/O operation has the same cost and set the operation cost constant to 1.

Considering that applications without I/O optimization may efficiently use computing resources but result in numerous sub-optimal smaller I/O operations for I/O and storage system resources, measuring the I/O intensity of an application becomes a valuable metric. This metric represents the ratio of I/O operations to the total data processed during the runtime of an application and is expressed as a unit of IOP/byte.

B. I/O Score based on the I/O Roofline Model

Traditionally, a score quantifies performance, such as in exams where it is linked to correct answers over total questions. In the context of quality, scores rely on indicators such as completion rate or work quality. Scores can be absolute or relative to a standard, with no unit of measurement specified. For performance evaluation, the I/O score provides a standardized, concise representation for relative performance comparison. It assists users in understanding the performance of their applications and identifying improvements. In addition, a score can be easily integrated with other tools and workflows, allowing users to automate performance monitoring and analysis. This is necessary due to the inconsistency and lack of scoring methods in III. Our approach enables the comparison of applications and systems in terms of their I/O performance.

1) *Scoring Application I/O Performance*: Our application performance score is based on the Euler distance, which is a measure of the distance between two points (P and Q) in a n-dimensional space and can be calculated as shown in Eq. 9:

$$\Delta(P, Q) = \sqrt{\sum_{\text{dim}=1}^n (p_i - q_i)^2} \quad (9)$$

Adjusting Euler distance for application performance evaluation to peak system performance is straightforward. We have previously mapped performance characteristics to a 2D plane via the Roofline model: intensity (IOP/byte) on the x-axis and maximum attainable peak performance (IOPS) on the y-axis. These coordinates serve for distance calculation, indicating the deviation of current performance from optimal. We employ logarithmic values rather than absolute values due to the scale's nature, ensuring alignment between the score and visual representation for improved clarity and accuracy. As detailed in Section II, the ideal system peak performance corresponds to the ridge point. The distance between the application performance and the ridge point for each dimension can be calculated using the following equation:

$$\Delta(P_{\text{ridge}}, P_{\text{app}}) = \sqrt{\sum_{\text{dim}=1}^n (\log_{10}(\frac{P_{\text{ridge}}}{P_{\text{app}}}))^2} \quad (10)$$

Using Equation 10 to calculate the distance offers a versatile approach applicable to any Roofline model without modification. This enables integration with existing models [6], [10] considering additional performance factors, promoting the development of a multidimensional score for the exploration of overlaps in computation, I/O, and communication. After computing distances in each dimension, we determine a relative value for each dimension, the partial score, ranging from 0 to 1. The partial score depends on similarity, calculated from the previously obtained distance values, as shown in Equation 11. The score increases as an application gets closer to the ridge point, signaling progress toward peak performance, and is calculated as the inverse of the distance.

$$\text{Score}_{\text{dim}} := \frac{1}{1 + \Delta(P_{\text{ridge}}, P_{\text{app}})} \quad (11)$$

In certain instances, the application may achieve optimal performance, resulting in a distance of zero. To avoid an exception that results in a division by zero, a value of one has been introduced. This adjustment ensures that the score remains within the desired range, even where the distance is not present. In conclusion, the final performance score is determined using Equation 12. Here, the weighted geometric mean of the values derived from the different performance dimensions is calculated, using positive weights $w_i > 0$ with $w := \sum_{\text{dim}=1}^n w_i$. The result leads to a comprehensive and unbiased presentation of the overall performance.

$$\overline{\text{Score}}_{\text{app}} := \left(\prod_{\text{dim}=1}^n \text{score}_i^{w_i} \right)^{\frac{1}{w}} = \sqrt[w]{\text{score}_i^{w_i}} \quad (12)$$

This evaluation approach allows the Roofline to provide a unified score, which in turn allows for a quick and consistent evaluation of the application's performance.

2) *Scoring System Performance*: With exponential data growth, storage system performance becomes increasingly vital for data-intensive scientific applications. Evaluating systems and their I/O subsystems in terms of I/O performance is paramount. Ranking approaches like the IO500 benchmark are gaining prominence, driving innovation in storage technology. As discussed in Section III, the IO500 score is a popular choice for evaluating the performance of I/O subsystems. This benchmark suite includes various test cases using IOR and mdtest, which require averaging the score. However, a problem arises from the different units of the benchmark results: IOR is in Mbps and mdtest in IOPS, which adds to the difficulty of deriving a meaningful score. To address this issue, we propose using the I/O Roofline model for ranking I/O subsystems, referencing the ridge point as the peak system performance. This approach is benchmark-independent, enabling precise performance assessment for each application and determining a system's maximum I/O potential.

When ranking HPC systems, the overall performance is usually given as an absolute value instead of a relative value [25], [31]. This has the advantage that not all point values have to be rewritten when updating the list and also directly reflects the actual performance. Thus the evaluation of a ridge point at the system level can be done in two ways:

- 1) **Vector-based score** is the simplest concept where the ridge point is represented as a vector, i.e., the score is a vector-based IOPS and I/O intensity (see Equation 13).

$$\text{Score}_{\text{sys}} := P_{\text{ridge}} = (\psi, \lambda)^T = \begin{pmatrix} \psi \\ \lambda \end{pmatrix} \quad (13)$$

- 2) **I/O bandwidth-based score** uses the product of peak IOPS and the inverse of I/O intensity as score and is defined in Equation 14. The score is given the unit byte/s.

$$\text{Score}_{\text{sys}} := P_{\text{ridge}} = \kappa \cdot (\lambda)^{-1} \quad (14)$$

The proposed scoring technique facilitates the evaluation and ranking of HPC systems based on their I/O performance in a consistent manner with a meaningful unit. A key ad-

vantage of this approach is the flexibility that allows the use of different benchmarks so that the I/O performance of the system can be evaluated for various workloads.

V. WORKFLOW FOR THE I/O ROOFLINE ANALYSIS

Our proposed I/O Roofline model defines four parameters, categorized as either *system-specific* or *application-specific*, as outlined in Table III. The approach for determining these parameters depends on their category. Peak performance determination involves two main approaches: *qualitative* and *quantitative*. Qualitative methods rely on theoretical analysis and complex models [6], while quantitative methods are based on empirical measurements, offering reliability and accuracy. The empirical approach requires measurement tools like profiling tools and hardware performance counters [32], however it is a practical choice for users with limited resources and experience. Therefore, our work focuses on establishing system-specific parameters for peak IOPS and I/O bandwidth through measurements from well-known community benchmarks.

Parameter	Dim.	Derivation	Specificity
Peak IOPS	$\frac{IOP}{s}$	Benchmark	System-specific
Peak I/O Bandwidth	$\frac{Byte}{s}$	Benchmark	System-specific
I/O Intensity	$\frac{IOP}{Byte}$	Characterization	Application-specific
I/O Bandwidth	$\frac{Byte}{s}$	Characterization	Application-specific

TABLE III: Parameters for the empirical I/O Roofline model.

The I/O intensity is an application-specific parameter that describes the relationship between the number of I/O operations and the amount of bytes read and written. It can be determined in two ways, by measurements or by analysis of the code. As the latter is not suitable for complex applications, we propose to derive the application-specific parameter from a low-overhead characterization tool instead of a tracing tool.

The application-specific parameter I/O bandwidth is determined by measurement, where profiling, tracing or characterization tools can be used. To ensure a consistent workflow and ensure optimal efficiency, we have chosen to use a characterization tool to determine the I/O bandwidth.

The workflow and Roofline model are tool-independent and adaptable to various analysis scenarios. In this study, we use IOR and Darshan for parameter derivation. IOR mimics diverse I/O patterns using interfaces such as HDF5, MPI-IO, and POSIX, which helps in evaluating I/O performance and is also used for IO500 ranking. Darshan is a commonly used tool that allows characterization of an application’s I/O behavior with minimal overhead. Support for profiling and characterization tools like Darshan allows versatile derivation of system-specific parameters from any application or benchmark.

Figure 2 illustrates the initial step in I/O Roofline analysis, involving benchmark and application I/O characterizations. Typically, IOR provides bandwidth and IOPS data for specific I/O patterns, offering directly applicable performance results. Alternatively, Darshan can be integrated with any application or benchmark to derive the necessary parameters. Unlike IOR,

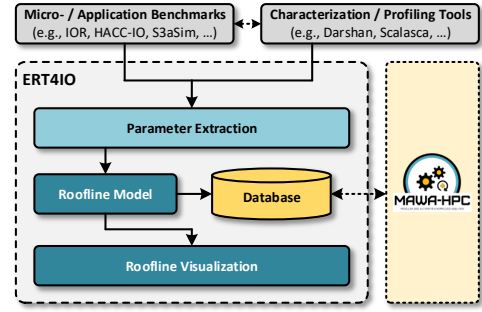


Fig. 2: Workflow of ERT4IO.

Darshan’s output requires filtering and aggregation of relevant data, such as I/O operations and data quantities.

Our workflow is automated and visualized using the Python tool *ERT4IO* (Empirical Roofline Tool for I/O) [33]. ERT4IO extracts system-specific and application-specific parameters based on specified output paths and interface types (e.g., POSIX or MPI-IO). It generates the corresponding Roofline model and visualization, which can be saved as an image file. Customizable scores can be generated if required and visually represented for specific use cases, such as application or system performance assessment.

Furthermore, we are developing the *Modular and Automated Workload Analysis for HPC* (MAWA-HPC) framework [7], [34]. It allows users to create and visualize I/O Roofline models by uploading Darshan output files through the user interface. The framework automatically extracts system- and application-specific parameters and then interactively displays the corresponding Roofline model on the web interface, providing a user-friendly method for evaluating I/O performance. Our goal is to store the generated Roofline model in a shared community database, facilitating various use cases, including system comparisons and evaluations across different configurations and hardware, and sharing insights with the HPC community.

VI. EXPERIMENTAL EVALUATION

To evaluate the applicability of the I/O Roofline model and the derived scoring approaches, case studies are conducted on the FUCHS-CSC with different benchmarks and applications.

A. Test Environment

The FUCHS-CSC cluster [35] consists of 198 nodes, each equipped with 2x Intel Xeon E5-2670 v2, providing 20 cores, 128 GB RAM, and FDR InfiniBand. The nodes can be allocated to provide a total of 3960 cores. FUCHS-CSC hosts a parallel scratch file system based on BeeGFS with an aggregated bandwidth of 27 GB/s and a capacity of 2.4 PB.

We use the NAS Parallel Benchmarks (NPB) [36] and Hardware Accelerated Cosmology Code (HACC) [37] as application kernels. IOR is considered as a reference and also applied to derive system-specific parameters i.e., peak I/O bandwidth and peak IOPS. HACC-IO uses N-body techniques to simulate the formation of structures in the universe and captures various I/O patterns, including checkpoint and restart,

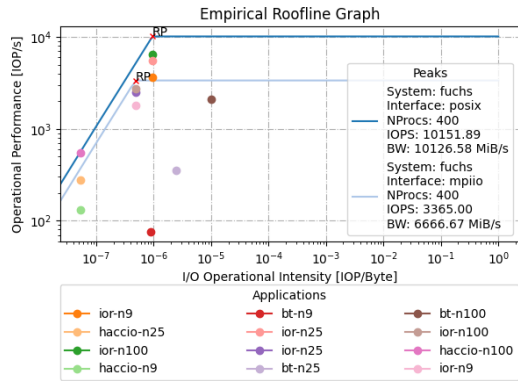


Fig. 3: Empirical Roofline analysis for FUCHS-CSC.

as well as file access patterns such as single-shared file or file-per-process. NPB consists of a set of programs commonly used to evaluate the performance of parallel supercomputers, derived from computational fluid dynamics applications. It includes 5 kernels and 3 pseudo-applications.

B. Applicability to the Evaluation of Applications

The study’s initial goal is to validate the accuracy of representing Darshan-characterized applications in the Roofline model and to prepare future support of various parallel I/O stack layers. We build models for the POSIX and MPI-IO interfaces. Notably, our study does not focus on determining the peak performance of the Fuchs-CSC cluster. We assume peak performance with 400 processes and IOR in the *IOR-easy* configuration (read and write operations per IO500). We utilize IOR, HACC-IO, and the I/O kernel for NPB’s tridiagonal block solver. To highlight the difference between peak and application performance, we conduct experiments with a maximum of 100 processes (-n100).

As illustrated in Figure 3, distinct Rooflines are generated for each I/O interface: light blue for MPI-IO and dark blue for POSIX. Peak performance for POSIX is 10126.58 MiB/s and 10151.89 IOPS, while MPI-IO achieves 6666 MiB/s and 3365 IOPS. Colored dots represent applications and IOR with fewer processes below their respective peak performance, showcasing varying I/O intensity due to different I/O patterns, problem sizes, and interfaces. To create the Roofline for both I/O interfaces, IOR is used. In addition, IOR runs with fewer processes using the same API are performed in parallel under the corresponding ridge points, so that the same I/O intensity is maintained despite different IOPS peaks. This occurs since the file-per-process pattern inflates the data volume proportionally to the number of processes, thus scaling the problem size proportionally. However, the number of I/O operations per second that the underlying resources can handle depends on the size of the problem. Consequently, different applications exhibit a spectrum of I/O intensity levels in the Roofline diagram. HACC-IO shows a similar pattern to IOR, scaling application workload with process count. In contrast, NPB’s I/O intensity grows with the number of processes due to its fixed problem size (Class B: 102x102x102 3D grid, 200 iterations), maintaining consistent read and write

Application (POSIX)	Score	Application (MPI-IO)	Score
ior-n9	0.83	ior-n9	0.89
ior-n25	0.89	ior-n25	0.94
ior-n100	0.91	ior-n100	0.96
haccio-n9	0.39	bt-n9	0.54
haccio-n25	0.41	bt-n25	0.54
haccio-n100	0.44	bt-n100	0.60

TABLE IV: Application score based the I/O Roofline model.

data amounts regardless of process count. NPB also employs MPI-IO collective I/O instead of file-per-process pattern, with measurement points below the MPI-IO interface’s peak performance due to collective I/O. These I/O variations underline the importance of considering unique application I/O patterns in performance evaluation. Furthermore, it should be noted that the I/O intensity varies for identical applications with different I/O interfaces, so that different numbers of operations are required for identical amounts of data.

To demonstrate the scoring methodology based on the I/O Roofline model, scores were computed for both I/O interfaces and all applications, as shown in Table IV. The table reveals that application scores depend on I/O intensity, IOPS, and distance to the peak ridge point. Application scores decrease with increasing I/O intensity and improve as I/O intensity decreases. Thus, applications with numerous minor I/O operations receive lower scores. This scoring method precisely maps application performance from the Roofline model to a score, enhancing overall assessment of application efficiency. Our case study showcases the I/O Roofline model’s effectiveness. System- and application-specific parameters are extracted using characterization tools like Darshan or benchmarks to inform I/O Roofline modeling. By mapping I/O performance of different applications onto the I/O Roofline model, our scoring approach effectively characterizes and compares their performance.

C. Applicability to the Evaluation of Systems

Next, we focus in particular on the suitability of the I/O Roofline model for evaluating different systems. We include Cesari as a reference system. Similar to the FUCHS-CSC cluster, Cesari also supports InfiniBand (FDR) and features 20 cores per node with dual socket Intel Xeon E5-2660 v2. However, Cesari is significantly smaller and lacks access to a parallel file system. Including Cesari allows us to assess the impact of a parallel file system on I/O performance. This comparison helps determine whether the parallel file system of FUCHS-CSC provides a significant advantage in I/O performance compared to a smaller system like Cesari.

Figure 4 compares the two test systems using our I/O Roofline model with identical I/O access patterns (as introduced in Section VI-B), employing up to 100 processes. The results reveal that the FUCHS-CSC cluster achieves peak performance (3333.33 MiB/s and 3416.5 IOPS) with a different number of processes than Cesari (1000 MiB/s and 1024 IOPS at minimum processes). FUCHS-CSC’s peak performance increases with more processes, while Cesari’s decreases. It is worth noting that Cesari lacks access to a

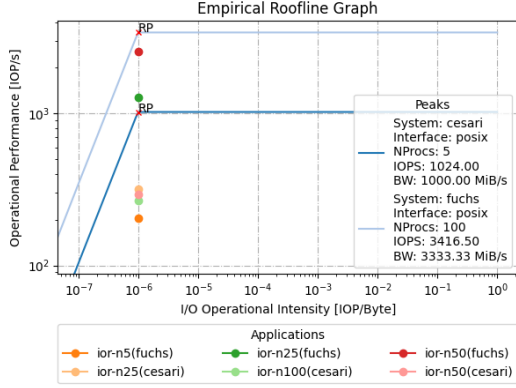


Fig. 4: Roofline model comparison for multiple systems.

parallel file system, which significantly impacts its performance, warranting further investigation. The IOR benchmark is executed with the same file-per-process I/O pattern on different systems, resulting in a workload that scales with the number of processes, as can be observed in the case of VI-B. Consequently, the measurement points that are executed with fewer processes are below peak performance, resulting in consistent I/O operational intensity for all iterations on both systems, with peak IOPS varying.

The systematic approach for evaluating I/O system performance involves deriving system-specific parameters from specific benchmarks that define the Roofline for each system, as mentioned in Section V. The Roofline representation enables evaluation based on the ridge point, which is defined by I/O intensity and IOPS. An absolute score for system evaluation can be derived based on two methods (refer to IV-B2), which greatly improves the comparability of I/O systems. In the vector-based scoring approach, the ridge point is transposed and represented as a multidimensional vector. As we focus exclusively on the I/O aspect of the systems, the results for FUCHS-CSC and Cesari can be calculated as shown in Equations 15 and 16, respectively.

$$\text{Score}_{\text{Fuchs}} = \begin{pmatrix} 3416.5 \\ 9.77 \cdot 10^{-7} \end{pmatrix} \begin{bmatrix} \text{IOP/s} \\ \text{IOP/Byte} \end{bmatrix} \quad (15)$$

$$\text{Score}_{\text{Cesari}} = \begin{pmatrix} 1024 \\ 9.77 \cdot 10^{-7} \end{pmatrix} \begin{bmatrix} \text{IOP/s} \\ \text{IOP/Byte} \end{bmatrix} \quad (16)$$

In the context of evaluating I/O performance through bandwidth-based scoring, we are concerned with multiplying the peak IOPS with the inverse of the I/O intensity. The results for both systems are shown in Equations 17 and 18:

$$3416 \left[\frac{\text{IOP}}{\text{s}} \right] \cdot \frac{1}{9.77 \cdot 10^{-7}} \left[\frac{\text{IOP}}{\text{Byte}} \right] \approx 3334 \left[\frac{\text{MiB}}{\text{s}} \right] \quad (17)$$

$$1024 \left[\frac{\text{IOP}}{\text{s}} \right] \cdot \frac{1}{9.77 \cdot 10^{-7}} \left[\frac{\text{IOP}}{\text{Byte}} \right] \approx 1000 \left[\frac{\text{MiB}}{\text{s}} \right] \quad (18)$$

Taking into consideration our scoring approach for system performance evaluation, the I/O performance of different systems can be illustrated and compared by means of their ridge points. During our assessment, Fuchs-CSC recorded a

peak of 3416.5 I/O operations per second, whereas Cesari only achieved approximately one-third of Fuchs-CSC’s peak performance. The peak bandwidth analysis also yielded a similar outcome. The results of both scoring approaches indicate that Fuchs-CSC performs significantly better compared to Cesari, highlighting the importance of a parallel file system in achieving optimal performance. In general, the Roofline visualization provides a straightforward and comprehensible way to represent the peak I/O performances of various systems. Both scoring approaches provide a consistent view of a system’s overall I/O performance by considering multiple dimensions at the same time. Hence, our evaluation method is beneficial when assessing and comparing systems with diverse architectures and configurations, as it facilitates the fair evaluation and comparison of their I/O performance in a consistent and reproducible manner.

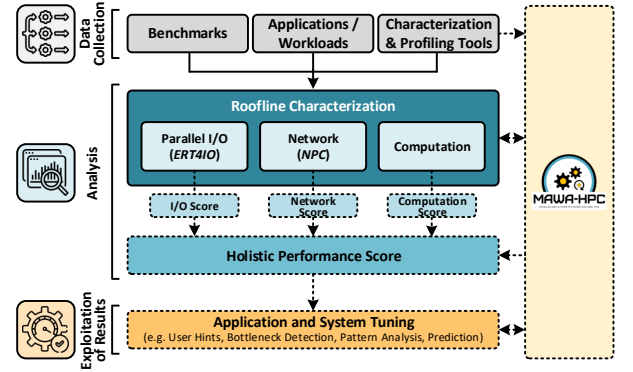


Fig. 5: MAWA-HPC Integration of the I/O Roofline model.

VII. OUTLOOK

Our preliminary I/O Roofline model and workflow hold potential for future improvement. To provide a more comprehensive view of system and application performance, we plan to introduce a multidimensional performance evaluation. This holistic performance score, as depicted in Figure 5, will consider factors like network e.g., by using *Network Performance Collector* (NPC) [38], compute power, and parallel I/O. It will automatically collect and integrate system- and application-specific parameters from various performance dimensions. Furthermore, ERT4IO has been integrated with our MAWA HPC framework, which enables comprehensive and interactive analysis for various use cases, including user hints such as Drishti [39], bottleneck detection, pattern analysis, and performance prediction. Based on the potential use cases, valuable insights can be archived to understand the performance of new workloads.

Finally, the I/O Roofline model and its workflow can be extended for more detailed system comparisons and application performance evaluations. This extension includes incorporating system characterization and weighted I/O operations for more fine-grained performance assessment, but also the validation against the IO500 list and other benchmarks. Ongoing enhancements involve integrating historical Darshan

logs to consider noise and concurrency as additional factors for Roofline analysis. Since our workflow is tool and platform agnostic, it can be extended to support additional profiling and tracing tools and benchmarks, including additional I/O interfaces like HDF5 [40] and PnetCDF [41].

VIII. CONCLUSION

In this paper, we proposed an empirical I/O Roofline model as a solution for complex performance modeling in HPC systems characterized by hardware heterogeneity and emerging workloads. Our model is based on the well-established Roofline model and provides an intuitive approach for various use cases. To simplify parameter extraction, we use ERT4IO, a Python-based tool that efficiently and automatically extracts the required parameters from different sources, including benchmarks and Darshan outputs. It also offers a convenient visualization of multiple rooflines and applications in a single diagram. We demonstrated the effectiveness of our Roofline model in characterizing I/O performance for different applications and assessing system performance. Furthermore, we propose to utilize the Roofline model as a multi-dimensional performance metric for system characterization in the future.

REFERENCES

- [1] S. Neuirth, "Modular Supercomputing and its Role in Europe's Exascale Computing Strategy," *PoS*, vol. LATTICE2022, p. 245, 2023.
- [2] S. W. Chien, A. Podobas, I. B. Peng, and S. Markidis, "tf-Darshan: Understanding fine-grained I/O performance in machine learning workloads," in *2020 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 359–370, IEEE, 2020.
- [3] H. Devarajan, H. Zheng, A. Kougkas, X.-H. Sun, and V. Vishwanath, "Dlio: A data-centric benchmark for scientific deep learning applications," in *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, pp. 81–91, IEEE, 2021.
- [4] J. L. Bez, S. Byna, and S. Ibrahim, "I/o access patterns in hpc applications: A 360-degree survey," *ACM Computing Surveys*, 2023.
- [5] S. Neuirth and A. K. Paul, "Parallel I/O Evaluation Techniques and Emerging HPC Workloads: A Perspective," in *2021 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 671–679, 2021.
- [6] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [7] Z. Zhu, S. Neuirth, and T. Lippert, "A Comprehensive I/O Knowledge Cycle for Modular and Automated HPC Workload Analysis," in *2022 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 581–588, IEEE, 2022.
- [8] P. Carns, J. Kunkel, K. Mohror, and M. Schulz, "Understanding I/O behavior in scientific and data-intensive computing (Dagstuhl Seminar 21332). Dagstuhl Rep. 11 (7), 16–75 (2021)."
- [9] D. Biswas, S. Neuirth, A. K. Paul, and A. R. Butt, "Bridging Network and Parallel I/O Research for Improving Data-Intensive Distributed Applications," in *2021 IEEE Workshop on Innovating the Network for Data-Intensive Science (INDIS)*, pp. 50–56, IEEE, 2021.
- [10] D. Cardwell and F. Song, "An extended roofline model with communication-awareness for distributed-memory hpc systems," in *Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region*, pp. 26–35, 2019.
- [11] "IOR Benchmark." <https://github.com/hpc/ior>. Accessed: 2023-01-28.
- [12] "Darshan I/O characterization tool." <https://github.com/darshan-hpc/darshan>. Accessed: 2023-03-13.
- [13] M. Harchol-Balter, *Performance modeling and design of computer systems: queueing theory in action*. Cambridge University Press, 2013.
- [14] G. Redinbo, "Queueing systems, volume i: Theory-leonard kleinrock," *IEEE Transactions on Communications*, vol. 25, no. 1, 1977.
- [15] J. Banks, *Discrete event system simulation*. Pearson Education, 2005.
- [16] H. H. Liu, *Software performance and scalability: a quantitative approach*. John Wiley & Sons, 2011.
- [17] S. Lee, J. S. Meredith, and J. S. Vetter, "Compass: A framework for automated performance modeling and prediction," in *Proceedings of the 29th ACM on International Conference on Supercomputing*, 2015.
- [18] Q. Wu and V. V. Datla, "On performance modeling and prediction in support of scientific workflow optimization," in *2011 IEEE World Congress on Services*, pp. 161–168, IEEE, 2011.
- [19] E. D. Lazowska, J. Zahorjan, G. S. Graham, and K. C. Sevcik, *Quantitative system performance: computer system analysis using queueing network models*. Prentice-Hall, Inc., 1984.
- [20] J. McCalpin, "Stream: Sustainable memory bandwidth in high performance computers," <http://www.cs.virginia.edu/stream/>, 2006.
- [21] S. Williams, L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel, "Optimization of sparse matrix-vector multiplication on emerging multicore platforms," in *Proceedings of the 2007 ACM/IEEE Conference on Supercomputing*, pp. 1–12, 2007.
- [22] S. Williams, J. Carter, L. Oliker, J. Shalf, and K. Yelick, "Lattice boltzmann simulation optimization on leading multicore platforms," in *2008 IEEE International Symposium on Parallel and Distributed Processing*, pp. 1–14, IEEE, 2008.
- [23] K. Datta, M. Murphy, V. Volkov, S. Williams, J. Carter, L. Oliker, D. Patterson, J. Shalf, and K. Yelick, "Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures," in *SC'08: Proceedings of the 2008 ACM/IEEE conference on Supercomputing*, pp. 1–12, IEEE, 2008.
- [24] M. Frigo and S. G. Johnson, "The design and implementation of fftw3," *Proceedings of the IEEE*, vol. 93, no. 2, pp. 216–231, 2005.
- [25] "IO500 Benchmark." <https://github.com/IO500>. Accessed: 2023-01-28.
- [26] R. Liem, D. Povaliaiev, J. Lofstead, J. Kunkel, and C. Terboven, "User-Centric System Fault Identification Using IO500 Benchmark," in *2021 IEEE/ACM Sixth International Parallel Data Systems Workshop (PDSW)*, pp. 35–40, IEEE, 2021.
- [27] F. Checconi, J. J. Tithi, and F. Petrini, "Ridgeline: A 2D Roofline Model for Distributed Systems," *arXiv preprint arXiv:2209.01368*, 2022.
- [28] S. Wu, H. Jiang, D. Feng, L. Tian, and B. Mao, "WorkOut: I/O Workload Outsourcing for Boosting RAID Reconstruction Performance," in *FAST*, vol. 9, pp. 239–252, 2009.
- [29] H. Devarajan, A. Kougkas, P. Challa, and X.-H. Sun, "Vidya: Performing code-block I/O characterization for data access optimization," in *2018 IEEE 25th International Conference on High Performance Computing (HiPC)*, pp. 255–264, IEEE, 2018.
- [30] X.-H. Sun and L. M. Ni, "Another view on parallel speedup," in *Proceedings of the 1990 ACM/IEEE conference on Supercomputing*, pp. 324–333, 1990.
- [31] "Top500 - The List." <https://www.top500.org/>. Accessed: 2023-01-30.
- [32] J. J. Dongarra, P. Luszczyk, and A. Petitet, "The linpack benchmark: past, present and future," *Concurrency and Computation: practice and experience*, vol. 15, no. 9, pp. 803–820, 2003.
- [33] Z. Zhu, N. Bartelheimer, and S. Neuirth, "An Empirical Roofline Model for Extreme-Scale I/O Workload Analysis," in *2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pp. 622–627, 2023.
- [34] Z. Zhu, N. Bartelheimer, and S. Neuirth, "MAWA-HPC: Modular and Automated Workload Analysis for HPC Systems." Research Poster, ISC High Performance Conference 2023 (ISC23), 2023.
- [35] "The CPU Cluster FUCHS-CSC." <https://csc.uni-frankfurt.de/wiki/doku.php?id=public:service:fuchs>. Accessed: 2023-01-29.
- [36] "NAS Parallel Benchmarks ." <https://www.nas.nasa.gov/software/npb.html>. Accessed: 2023-01-29.
- [37] "HACC I/O ." <https://github.com/glennklockwood/hacc-io/>. Accessed: 2023-01-29.
- [38] N. Bartelheimer, Z. Zhu, and S. Neuirth, "Toward a Modular Workflow for Network Performance Characterization," in *2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pp. 331–334, 2023.
- [39] J. L. Bez, H. Ather, and S. Byna, "Drishti: Guiding End-Users in the I/O Optimization Journey," in *2022 IEEE/ACM International Parallel Data Systems Workshop (PDSW)*, pp. 1–6, 2022.
- [40] "The HDF Group - Hierarchical Data Format, version 5." <https://www.hdfgroup.org/solutions/hdf5/>. Accessed: 2022-07-3.
- [41] J. Li, W.-k. Liao, A. Choudhary, R. Ross, R. Thakur, W. Gropp, R. Latham, A. Siegel, B. Gallagher, and M. Zingale, "Parallel netCDF: A high-performance scientific I/O interface," in *SC'03: Proceedings of the 2003 ACM/IEEE Conference on Supercomputing*, pp. 39–39, 2003.